cw2 weak

Machine Learning for Computer Vision Coursework 2

Lida Jin (CID: 00835494) & Tianyou Wang (CID: 00846075)
Department of Electrical and Electronic Engineering
Imperial College London

1. Matching

1.1 Manual Matching

First, we select 10 interest points in both images manually, shown in Figure 1 below.



Figure 1 Manual Selection of Interest Points

1.2 Automatic Matching

Harris–Stephens algorithm is used for automatic interest point detection. This method identifies the corner points, for example, corners of the documents in the shelf, as shown in Figure 2 below. Due to different angle of the images, different numbers of interest points are detected. We can see a cluster of interest points in the top right part of Image 2, which are excluded from Image 1. In this case, 242 and 254 corner points for Image 1 and 2 respectively.



Figure 2 Automatic Feature Detection (Harris)

Simple Sq[illegible] Neighbourhood method is used to find matching of descriptors. From Figure 3 below, we can see that the matching was accurate. Another observation is there are significantly less descriptors than the interest points dete[illegible]n the previous section. This is because some points are not valid as it does not appear in both pictures and others are redundant due to repetition in vicinity.



Figure 3 [illegible]st Neighbour Matching of Descriptors

1.3 Transformation Estimation

1.3.1 Homography Matrix and Fundamental Matrix

The Homography Matrix (M) is defined as:

$$u = \mathbf{M}x$$

Where x is a vector containing original coordinates and u is the vector of projected coordinates.

The Matlab function *fitgeotrans* is used to find the homography matrix. It takes in the matched descriptors from previous section as inputs to find the transformation properties. An output of class *projective2d* is generated which contains a 3×3 forward 2-D projective transformation matrix (T):

$$x^T = u^T\mathbf{T}$$

We can therefore find the Homography Matrix (M) by:

$$\mathbf{M} = (\mathbf{T}^T)^{-1}$$

The fundamental matrix is calculated using the *estimateFundamentalMatrix* function which takes in the matched corner points for the two images.

1.3.2 Homography Accuracy

We can find the coordinates of projected points by applying the homography matrix to the points in Image 1. This is done by the *transformPointsForward*

Summary of Comments on Microsoft Word - cw2_TW2813_LJ1713.docx

Page: 1

Number: 1 Author: . Subject: Sticky Note Date: 19/05/2017 12:23:02

General: ok formatting, images can be better (overlay points over image). Almost no own code, only matlab implementation. Not good discussion, and last section is almost empty. No own images used.
1) Not clear is Nearest Neighbour is used (not in the code also). Not clear what descriptor is used. Examples of matched points given.
2) No function implemented, all from MATLAB. Figure 4 difficult to see. Automatic method gives quite higher HA than manual method.
3) The scaled images are not compared, instead scaled pair is compared to original pair (two different HA). The comparisons are not really good, and the automatic method should perform better.
4) Only disparity map, and not really in-depth. The rest of sections are empty.

Number: 2 Author: . Subject: Sticky Note Date: 18/05/2017 19:13:57

Ok, manual matching example.

Number: 3 Author: . Subject: Sticky Note Date: 18/05/2017 19:15:04

Here it is said Nearest Neighbour, I don't see in the code what it was actually done. Probably it is Nearest Neighbour then.

Number: 4 Author: . Subject: Sticky Note Date: 18/05/2017 19:16:45

It would have been good to do your own implementation.

Number: 5 Author: . Subject: Sticky Note Date: 18/05/2017 19:13:40

Its not square neighbourhood, its Nearest Neighbour between descriptors. Square Neighbourhood will only match points close in both images (it does not work for scaling, or rotation...)

Number: 6 Author: . Subject: Sticky Note Date: 18/05/2017 19:18:01

What descriptors are used? Histograms?

function. The 10 manually selected interest points are projected and shown below in Figure 4 (*left*), and we can see the error is relatively small.

The accuracy of projection found by calculating the average Euclidean distance in number of pixels between original points and projected points. The equation of Euclidean distance is:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

For the manually selected interest points, the error is 1.62.

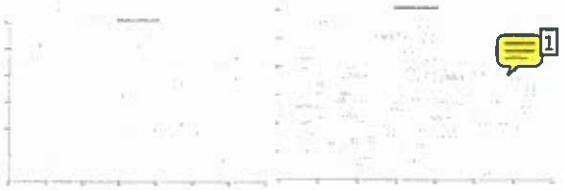


Figure 4 Difference between Projected Points and Actual Points

For the automatic detected features, the difference is shown in Figure 4 (*right*). The error is greater than manually selected features at 7.21. This could be due to less accurate automatic interest point detection compared to hand selection. Also, there are more points used for finding homography where automatic detection is used, compared to only 10 points selected manually.

1.3.3 Epipolar Line

Epipolar Line can be found directly by *epipolarLine* function. Figure 5 shows the Epipolar Lines for the manually selected interest points in both images.



Figure 5 Epipolar Lines

2. Image Geometry

2.1 Homography

2.1.1 HA Error and Size of Image

Figure 6 below shows the original image (*left*) and the scaled version (*right*) which is reduced by a factor of 2. The strongest 20 interest points detected is shown for both image in Figure 7, where we can see the interest points in the two images are quite different. The HA Error for the smaller image is 3.68, compared to 7.21 for the original image. The difference is because HA Error is defined in terms of pixels, and there are only a quarter of pixels in the smaller picture. Therefore, the error distance will be scaled down accordingly. We can multiply the HA Error of the smaller image by 2, which is 7.36, to get a fair comparison between the two cases.

Now, the error of smaller image is larger than original image by 2.1% . This is because with a smaller resolution, the corner points are less accurately defined. It is more difficult to find corner points in a blurry picture than one with high resolution. Also, in a high definition picture it is easier to pin-point the location of corner points, whereas in the scaled image, 1 pixel covers the area of 4 pixels in the original image.



Figure 6 Original (left) and Scaled Image (right)



Figure 7 Interest points of Original Image (left) and Interest Points of Scaled Image (right)

2.1.2 Compare Geometric Transformation Parameters

Geometric transformation parameters include: translation (t), scale (s), shear (sh), rotation (r) and tilt (e, f). This information can be derived from the forward 2-D projective transformation matrix ($\mathbf{T}$):

$$\mathbf{T} = \begin{bmatrix} s_x & sh_y & e \\ sh_x & s_y & f \\ t_x & t_y & 1 \end{bmatrix}$$

and r can be found by $cos^{-1}(s_x)$.

The results for both automatic and manual selection is summarised in Figure 8 below. We can see for both pictures, the main transformation is translation, and the largest difference is between the x-displacement values. Both case show a scaling of 1, which means objects in both images have same size. Additionally, it has no shearing and tilt with little rotation. We compared the percentage difference between automatic and manual in Figure 8 below.

	Automatic	Manual
x-displacement	-0.7	-17.7
y-displacement	1.2	6.9
x-scale	1.0	1.0

Page: 2

Number: 1 Author: . Subject: Sticky Note Date: 18/05/2017 19:17:23
Difficult to visualize.

Number: 2 Author: . Subject: Sticky Note Date: 18/05/2017 19:19:22
Again, own implementation would have been better.

Number: 3 Author: . Subject: Sticky Note Date: 19/05/2017 12:58:20
The results make sense, and the table is good. But further analysis is required. Also, only Matlab functions applied. The projected images could have been added, or testing more transformations, other descriptors, etc...

Number: 4 Author: . Subject: Sticky Note Date: 18/05/2017 19:20:25
The HA is computed for one scaled image against that same original image, hence only 1 HA. Not original pair vs scaled pair.

y-scale	1.0	1.0
x-shear	-0.0	0.0
y-shear	-0.0	-0.0
rotation	0.3	0.1
vanish-1	-0.0	-0.0
vanish-2	0.0	0.0

Figure 8 Comparison of Geometric Transformation Parameters

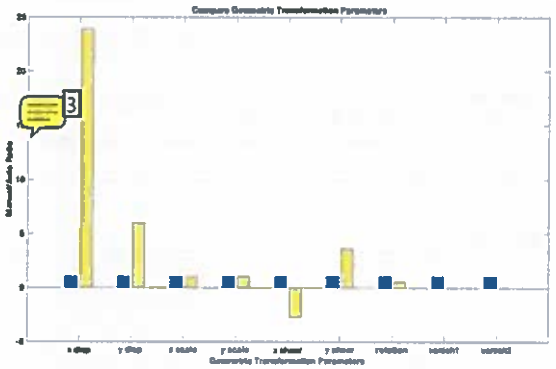


Figure 9 Comparison of Geometric Transformation Parameters

2.1.3 Different Number of Correspondences

The minimum required number of correspondences for estimating a homography matrix is 4 pairs of points, and the maximum available is 113 in this case. A homography matrix is estimated with each number of correspondences, and the matrix is applied to project points for comparison and calculation of HA error, the results are shown below in Figure 10. The error decreases sharply for small number of matched features. The minimum error of 3.38 occurs at 11 pairs of matched features, which is significantly lower than when all pairs are used.

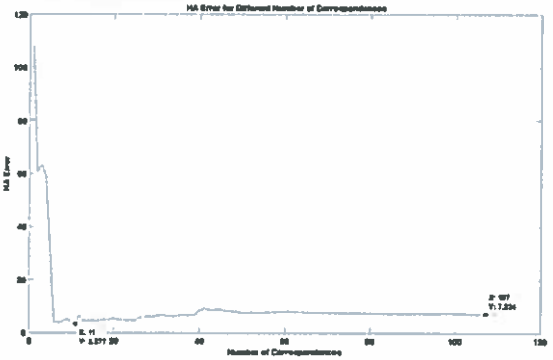


Figure 10 HA Error for Different Number of Correspondences

Figure 11 below visualises the first six instances of the above graph. Which is from 4 to 9 pairs of matched points for estimation of the homography matrix. The blue points show the projected points whereas the red ones are the actual ones. We can see that in the first graph the performance is very poor and quickly become reasonably accurate in the last diagram.

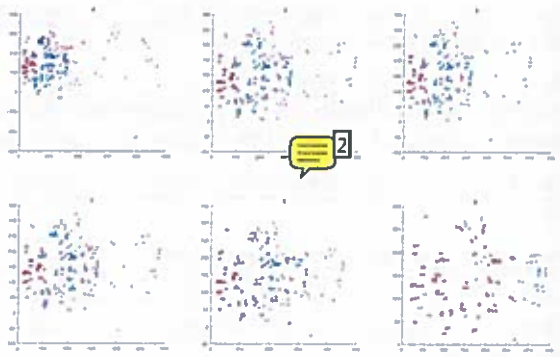


Figure 11 Visualisation of Projecting Points with Different Number of Correspondences

2.2 Stereo Vision

2.2.1 Estimate Fundamental Matrix

2.2.2 Epipoles and Epipolar Lines

2.2.3 Disparity Map and Depth Map

Disparity map is based on the principle that closer objects have greater offset in a stereo image, and vice versa. The disparity map is calculated from the two images and a disparity range is set. Figure 12 below shows the depth map with different disparity ranges, these values must be multiples of 16. The images below show closer objects as lighter and further darker. We can see a disparity range of [0: 16] can show the relative distance most apparently.



Figure 12 Depth Map with Different Disparity Range

2.2.4 Depth Map

2.2.5 Change focal length, add noise

Page: 3

Number: 1 Author: . Subject: Sticky Note Date: 18/05/2017 19:33:54
This is a table, not a figure.

Number: 2 Author: . Subject: Sticky Note Date: 18/05/2017 19:28:16
Hard to visualize. Better to overlay in the input images.

Number: 3 Author: . Subject: Sticky Note Date: 18/05/2017 19:30:35
It would have been better to discuss the values instead of this plot. Anyway, the automatic method HA should be better compared to manual. Other transformations could have been tested.

Number: 4 Author: . Subject: Sticky Note Date: 18/05/2017 19:31:05
Good.

Number: 5 Author: . Subject: Sticky Note Date: 18/05/2017 19:31:46
Not really useful, just changes the visualization intensity. Block size is a better parameter to change.

Number: 6 Author: . Subject: Sticky Note Date: 18/05/2017 19:31:51
Where is the rest?

Number: 7 Author: . Subject: Sticky Note Date: 18/05/2017 19:30:44
What about the outliers?

Appendix - Code Implementation

```
clear;clc
close all

%% Test images

img1_in = imread('1.ppm');
img2_in = imread('2.ppm');

figure
imshow(img1_in)
%
% imshow(img2)
%
figure
imshowpair(img1_in,img2_in,'falsecolor','ColorChannels','red-cyan')

%% Q1.1 a) Manual interest points
% imshow(img1_in)
% [point1_m(:,1),point1_m(:,2)] = ginput;
%
% figure
% imshow(img2_in)
% [point2_m(:,1),point2_m(:,2)] = ginput;
%%

load ginput2.mat
matchedPoints1_m = cornerPoints(point1_m);
matchedPoints2_m = cornerPoints(point2_m);

% Visualise match
figure
showMatchedFeatures(img1_in,img2_in,matchedPoints1_m,...
    matchedPoints2_m,'falsecolor','PlotOptions',{'r*','gx','y-'});
title('Manual Selection of Interest Points');
%% Q1.2 a) Automatic interest point detector - Harris

img1 = rgb2gray(img1_in);
img2 = rgb2gray(img2_in);

point1_a = detectHarrisFeatures(img1);
point2_a = detectHarrisFeatures(img2);

figure
subplot(1,2,1)
imshow(img1)
hold on
plot(point1_a)
hold off
title('Image 1');

subplot(1,2,2)
hold on
imshow(img2)
plot(point2_a)
hold off
title('Image 2');

%% Q1.2 b) Find descriptor matrices (feat1 and feat2)
```

```
[feat1,point1_val] = extractFeatures(img1,point1_a,'Method','Block');
[feat2,point2_val] = extractFeatures(img2,point2_a,'Method','Block');


figure
subplot(1,2,1)
imshow(feat1)
subplot(1,2,2)
imshow(feat2)
%% Q1.2 c) Nearest Neighbour matching

indexPairs = matchFeatures(feat1,feat2);

matchedPoints1 = point1_val(indexPairs(:,1),:);
matchedPoints2 = point2_val(indexPairs(:,2),:);

% Visualise match
figure
showMatchedFeatures(img1,img2,matchedPoints1,matchedPoints2,'falsecolor');
title('Nearest Neighbour Matching of Descriptors');
%% Q1.3 a) Estimate Homography matrix (homography included in tform)
%
% temp(:,:,1) = matchedPoints1.Location;
% temp(:,:,2) = matchedPoints2.Location;
% % homography = homest2d(temp);
% % homography = (1/homography(3,3)).*homography;
% hmatrixauto1 = autohomest2d(temp);

% homography matrix of auto features
tform = fitgeotrans(matchedPoints1.Location,...
    matchedPoints2.Location,'projective');

% homography matrix of manual points
tform_m = fitgeotrans(point1_m,point2_m,'projective');

hmat = inv(tform.T');
hmat = hmat./hmat(3,3);

hmat_m = inv(tform_m.T');
hmat_m = hmat_m./hmat_m(3,3);
%% Q1.3 b) Estimate Fundamental matrix

% fundamental_m = estimateFundamentalMatrix(matchedPoints1_m,...
%                                           matchedPoints2_m);
fundamental = estimateFundamentalMatrix(matchedPoints1,matchedPoints2);

%% Q1.3 c) Homography accuracy - Manual
% Two pictures A and B
% B is a projective transformation of A
% Estimate homography matrix given A and B
% Project B to a new image C using homography
% Calculate difference between C and A

original_m = point1_m;
projected_m = transformPointsForward(tform_m,point1_m);

% figure
% showMatchedFeatures(img1,img2,original_m,projected_m,'montage');
% title('Projected points');
```

```
figure
hold on
scatter(point2_m(:,1),point2_m(:,2),'bx')
scatter(projected_m(:,1),projected_m(:,2),'rx')
hold off
title('Projected points and Actual points');

ha_m = mean(sqrt((projected_m(:,1)-point2_m(:,1)).^2+(projected_m(:,2)...
    -point2_m(:,2)).^2));

%% Q1.3 c) Homography accuracy - Auto

original = matchedPoints1.Location;
point2 = matchedPoints2.Location;

projected = transformPointsForward(tform,point2);

figure
hold on
scatter(point2(:,1),point2(:,2),'bx')
scatter(projected(:,1),projected(:,2),'rx')
hold off
title('Projected points and Actual points');

ha = mean(sqrt((projected(:,1)-point2(:,1)).^2+(projected(:,2)...
    -point2(:,2)).^2));

%% Q1.3 d) Epipolar Line - Manual

epiLines1_m = epipolarLine(fundamental',point1_m);
epiLines2_m = epipolarLine(fundamental,point2_m);

p1_m = lineToBorderPoints(epiLines1_m,size(img1));
p2_m = lineToBorderPoints(epiLines2_m,size(img2));

figure;
subplot(1,2,1);
imshow(img1);
title('Epipolar Lines in First Image');
hold on
plot(point1_m(:,1),point1_m(:,2),'go')
line(p1_m(:,[1,3])',p1_m(:,[2,4])');
% truesize;
hold off

subplot(1,2,2);
hold on
imshow(img2);
title('Epipolar Lines in Second Image');
plot(point2_m(:,1),point2_m(:,2),'go')
line(p2_m(:,[1,3])',p2_m(:,[2,4])');

hold off

%% Q1.3 d) Epipolar Line - Auto

epiLines1 = epipolarLine(fundamental',original);
epiLines2 = epipolarLine(fundamental,point2);
```

```
p1 = lineToBorderPoints(epiLines1,size(img1));
p2 = lineToBorderPoints(epiLines2,size(img2));

figure;
subplot(1,2,1);
imshow(img1);
title('Epipolar Lines in First Image');
hold on
plot(original(:,1),original(:,2),'go')
line(p1(:,[1,3])',p1(:,[2,4])');
% truesize;
hold off

subplot(1,2,2);
hold on
imshow(img2);
title('Epipolar Lines in Second Image');
plot(point2(:,1),point2(:,2),'go')
line(p2(:,[1,3])',p2(:,[2,4])');
% truesize;
hold off

%% Q2.1 a) Interest point with different size - manaul selection of match


img1_small = imresize(img1,0.5);
img2_small = imresize(img2,0.5);

figure
imshowpair(img1,img1_small,'montage')
title('Original and Scaled Image');

point1_a_small = detectHarrisFeatures(img1_small);
point2_a_small = detectHarrisFeatures(img2_small);

figure
subplot(1,2,2)
imshow(img1_small)
hold on
plot(point1_a_small.selectStrongest(20))
hold off
title('Interest Points of Scaled Image');

subplot(1,2,1)
hold on
imshow(img1)
plot(point1_a.selectStrongest(20))
hold off
title('Interest Points of Original Image');

truesize;
%% Q2.1 a) Interest point with different size - auto selection of match

[feat1_s,point1_val_s] =
extractFeatures(img1_small,point1_a_small,'Method','Block');
[feat2_s,point2_val_s] =
extractFeatures(img2_small,point2_a_small,'Method','Block');

indexPairs_s = matchFeatures(feat1_s,feat2_s);
```

```
matchedPoints1_s = point1_val_s(indexPairs_s(:,1),:);
matchedPoints2_s = point2_val_s(indexPairs_s(:,2),:);

figure
subplot(1,2,1)
imshow(img1_small)
hold on
plot(matchedPoints1_s)
hold off

subplot(1,2,2)
hold on
imshow(img1)
plot(matchedPoints1)
hold off

truesize;
%% Compare HA

tform_s = fitgeotrans(matchedPoints1_s.Location,...
    matchedPoints2_s.Location,'projective');

original_s = matchedPoints1_s.Location;
point2_s = matchedPoints2_s.Location;

projected_s = transformPointsForward(tform_s,point2_s);

figure
hold on
scatter(point2_s(:,1),point2_s(:,2),'bx')
scatter(projected_s(:,1),projected_s(:,2),'rx')
hold off

ha_s = mean(sqrt((projected_s(:,1)-point2_s(:,1)).^2+(projected_s(:,2)...
    -point2_s(:,2)).^2));

% Multiply by 2 to get same comparison

%% Q2.1 b) Compare geometric transformation parameters

[tx,ty,sx,sy,shx,shy,q,e,f] = geo_param(tform.T);
[tx_m,ty_m,sx_m,sy_m,shx_m,shy_m,q_m,e_m,f_m] = geo_param(tform_m.T);

param  = [tx,tx_m;ty,ty_m;sx,sx_m;sy,sy_m;shx,shx_m;shy,...
          shy_m;q,q_m;e,e_m;f,f_m];

bar(param)

xticks(1:9);
xticklabels({'x disp','y disp','x scale','y scale',...
             'x shear','y shear','rotation','vanish1','vanish2'});
set(gca,'fontsize',18,'FontWeight','bold')
title('Compare Geometric Transformation Parameters','FontSize',
18,'FontWeight','bold')
xlabel('Geometric Transformation Parameters','FontSize', 18,'FontWeight','bold')
ylabel('Manual/Auto Ratio', 'FontSize', 18,'FontWeight','bold')


figure
hold on
```

```
plot(param(:,1))
plot(param(:,2))
hold off
%% Q2.1 c) HA for different number of points

% at least four points
i = 4;
ha_p = nan(113-3,1);
for i = 4:113
    tform_p = fitgeotrans(matchedPoints1.Location(1:i,:),...
        matchedPoints2.Location(1:i,:),'projective');

    % original = matchedPoints1.Location;
    % point2 = matchedPoints2.Location;

    projected = transformPointsForward(tform_p,point2);

    if i-3 <= 6
        subplot(2,3,i-3)
        hold on
        scatter(point2(:,1),point2(:,2),'bx')
        scatter(projected(:,1),projected(:,2),'rx')
        hold off
        title(i);
    end


    ha_p(i-3) = mean(sqrt((projected(:,1)-point2(:,1)).^2+(projected(:,2)...
        -point2(:,2)).^2));
end

figure
plot(ha_p)

set(gca,'fontsize',18,'FontWeight','bold')
title('HA Error for Different Number of Correspondences','FontSize',
18,'FontWeight','bold')
xlabel('Number of Correspondences','FontSize', 18,'FontWeight','bold')
ylabel('HA Error', 'FontSize', 18,'FontWeight','bold')


%% Q2.2 a) Fundamental matrix
%% Q2.2 b) Epipolar line and Epipoles

[isIn,epipole] = isEpipoleInImage(fundamental,size(img1));

%% Q2.2 c) Disparity map
figure
imshow(stereoAnaglyph(img1,img2));
title('Red-cyan composite view of the stereo images');

figure
for i = 1:4

    disparityRange = [0 16*i];
    disparityMap = disparity(img1,img2,'DisparityRange',disparityRange);

    subplot(2,2,i)

    imshow(disparityMap,disparityRange);
```

```
    %     title('Disparity Map');
    title(strcat('Disparity range: 0 - ',num2str(i*16)))
    colormap jet
    colorbar

end

figure
disparityRange = [0 16];
disparityMap = disparity(img1,img2,'DisparityRange',disparityRange);

% subplot(2,2,i)
imshow(disparityMap,disparityRange);
title('Disparity Map');
colormap jet
colorbar


%% Rectification

[fMatrix, epipolarInliers, status] = estimateFundamentalMatrix(...
    matchedPoints1, matchedPoints2, 'Method', 'RANSAC', ...
    'NumTrials', 10000, 'DistanceThreshold', 0.1, 'Confidence', 99.99);

if status ~= 0 || isEpipoleInImage(fMatrix, size(img1)) ...
        || isEpipoleInImage(fMatrix', size(img2))
    error(['Either not enough matching points were found or '...
        'the epipoles are inside the images. You may need to '...
        'inspect and improve the quality of detected features ',...
        'and/or improve the quality of your images.']);
end

inlierPoints1 = matchedPoints1(epipolarInliers, :);
inlierPoints2 = matchedPoints2(epipolarInliers, :);

figure;
showMatchedFeatures(img1, img2, inlierPoints1, inlierPoints2);
legend('Inlier points in I1', 'Inlier points in I2');

%%
[t1, t2] = estimateUncalibratedRectification(fMatrix, ...
    inlierPoints1.Location, inlierPoints2.Location, size(img2));
tform1 = projective2d(t1);
tform2 = projective2d(t2);

[I1Rect, I2Rect] = rectifyStereoImages(img1, img2, tform1, tform2);
figure;
imshow(stereoAnaglyph(I1Rect, I2Rect));
title('Rectified Stereo Images (Red - Left Image, Cyan - Right Image)');

%%  Disparity after rectification

figure
imshow(stereoAnaglyph(I1Rect,I2Rect));
title('Red-cyan composite view of the stereo images');

disparityRange = [-16 16];
disparityMap = disparity(I1Rect,I2Rect,'DisparityRange',disparityRange);

figure
```

```
imshow(disparityMap,disparityRange);
title('Disparity Map');
colormap jet
colorbar

%% Q2.2 e) Disparity with different focal length
%% Q2.2 e) Disparity with noise

%%
% close all
```